Week 9 Lab: Checkpoint 2 — End-to-End Integration Test

Difficulty: Intermediate

Lab Objectives

This week's lab is your capstone **Checkpoint 2** submission. There is no separate standalone project — your homework is to run your capstone system end-to-end and document the results.

By the end of this lab, you will have:

1. Run at least one record through all four components without manual intervention
2. Documented what works and what breaks
3. An updated capstone audit report with current project status
4. An updated `copilot-instructions.md` reflecting your project's current state
5. A prompt log with at least 5 entries

> ***Why this format:*** *Checkpoint 2 is the most important group milestone before final polish. A separate lab assignment would compete with this work for your time. Instead, this lab wraps the checkpoint deliverables into a structured submission so you get credit for the integration work you need to do anyway.*

Part 1: The End-to-End Test

What "end-to-end" means

One record enters your system through Ingestion and flows through every component — AI Core processes it, Specialist acts on it, and the result is visible in Integration's dashboard — **without you manually copying data between steps**.

This doesn't mean everything has to be perfect. It means the pipeline *runs*. If AI Core misclassifies the record, that's a quality issue to fix later. If the record never reaches AI Core because the status field is misspelled, that's an integration gap to fix now.

Step 1: Prepare a Test Record

Create one clean test record that represents a realistic input for your system. Write it down — you'll reference it throughout.

- **What it is:** [describe the record]
- **Expected path:** Ingestion writes it → AI Core processes it → Specialist acts on it → visible in dashboard
- **Expected final state:** [what should the record look like when it's done?]

Step 2: Run It

1. Trigger your Ingestion workflow with the test record
2. Watch it flow through each component
3. **Take a screenshot at each stage:**
    - Record in Airtable after Ingestion writes it (status should be `unprocessed` or equivalent)
    - Record after AI Core processes it (status advanced, AI fields populated)

- Record or new entry after Specialist acts on it
- Record visible in Integration's dashboard view

Step 3: Document What Happened

Create a file called `docs/checkpoint2-results.md` in your capstone repo:

```
# Checkpoint 2 Results

**Date:** [YYYY-MM-DD]
**Team:** [team name]
**Test record:** [describe what you sent through the pipeline]

## End-to-End Status: [PASSED / PARTIAL / FAILED]

## Component-by-Component Results

### Ingestion
- **Status:** [Working / Partially Working / Not Working]
- **What happened:** [describe — did the record land in Airtable correctly?]
- **Screenshot:** [reference screenshot filename]

### AI Core
- **Status:** [Working / Partially Working / Not Working]
- **What happened:** [did it pick up the record? Did it produce correct output?]
- **Screenshot:** [reference screenshot filename]

### Specialist
- **Status:** [Working / Partially Working / Not Working]
- **What happened:** [did it fire on AI Core's output? Did it produce the expected
action?]
- **Screenshot:** [reference screenshot filename]

### Integration Dashboard
- **Status:** [Working / Partially Working / Not Working]
- **What happened:** [is the completed record visible in the dashboard view?]
- **Screenshot:** [reference screenshot filename]

## Gaps Found
- [List every issue: field mismatches, status typos, workflows that didn't trigger,
missing fields, etc.]
- [For each gap, note which component/team member owns the fix]

## Fix Plan
1. [Highest priority fix — what, who, estimated effort]
2. [Next fix]
3. [Next fix]
```

> ***If the test fails partway through:*** *That's expected and fine. Document exactly where it broke and why. A clear failure report is a successful Checkpoint 2 — the goal is to find the gaps, not to have zero gaps.*

Part 2: Re-Run the Capstone Audit

Open Copilot Chat in VS Code (with your capstone repo open) and run the audit prompt from the Week 8 lab again. Answer all 10 questions based on your **current** project state — not your Week 8 answers.

Save the new report as `docs/checkpoint2-audit.md`, replacing the Week 8 version.

Compare the two: did the status change? Were the critical gaps from Week 8 addressed?

Part 3: Update Your AI-Assisted Dev Artifacts

Update `copilot-instructions.md`

Your project has changed since Week 8. Update the **Current State** section at minimum:

- What's working now that wasn't before?
- What new issues did Checkpoint 2 reveal?
- Has the schema changed?

Prompt Log: 5 Entries

Your `prompt-log-[yourname].md` should have at least **5 entries** by now. If you're short, this week's work gives you natural entries:

- Using Copilot to debug a Checkpoint 2 failure
- Asking Copilot to help fix a field name mismatch
- Generating test data for the integration test
- Debugging an n8n expression that broke during end-to-end flow
- Re-running the audit prompt and comparing results

Deliverables Checklist

All deliverables go in your capstone repo:

- [ ] **`docs/checkpoint2-results.md`** — end-to-end test documentation with component-by-component results
- [ ] **Screenshots** — one per component stage (Ingestion → AI Core → Specialist → Dashboard)
- [ ] **`docs/checkpoint2-audit.md`** — updated audit report (re-run from Week 8)
- [ ] **`.github/copilot-instructions.md`** — updated to reflect current project state
- [ ] **`prompt-log-[yourname].md`** — at least 5 entries (individual)

Grading Rubric

Component	Points	Requirements
End-to-end test attempted	20	Evidence that you ran a record through the pipeline (screenshots at each stage)
Checkpoint 2 results documented	25	Complete `checkpoint2-results.md` with honest component-by-component assessment and gap list

Fix plan	10	Prioritized list of gaps with owners — shows you know what to do next
Capstone audit re-run	15	Updated `checkpoint2-audit.md` reflecting current state, not a copy of Week 8
`copilot-instructions.md` updated	15	Current State section reflects post-Checkpoint 2 reality
Prompt log (5+ entries)	15	Individual log with context, evaluation, and reflection — not backfilled
Total	**100**	

Looking Ahead: Week 10

Next week covers **Security Orchestration & Playbook Design** — concepts that map directly to your capstone's detection → analysis → response flow. Your homework will focus on Phase 4 capstone work: error handling, confidence-based routing, and building your production dashboard. Use the fix plan from this week as your starting point.